

Applied Machine Learning Lab

B.Tech Semester 4

Experiment 08

Credit Score Classification using PCA and Machine Learning Models

Date: 25th Feb 2026

SAP ID: 590016154

Name: Vidisha

AIM

To analyze a credit score dataset, apply PCA for dimensionality reduction, and build classification models to predict credit scores.

THEORY

Credit score classification is a supervised learning problem.

Data Preprocessing: Handling missing values and encoding data

PCA: Reduces dimensionality while preserving variance

Models: Logistic Regression, Decision Tree, Random Forest, KNN, Naive Bayes

Evaluation Metrics: Accuracy, Precision, Recall, F1-score

Commands/Code Used

```
# Import libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
# Load dataset
```

```
df = pd.read_csv("train.csv")
```

```
# -----
```

```
# PREPROCESSING
```

```
# -----
```

```

# Handle missing values

df = df.fillna(method='ffill')


# Convert numeric columns (if needed)
for col in df.columns:

    try:

        df[col] = pd.to_numeric(df[col])

    except:

        pass


# Encode categorical columns

le = LabelEncoder()

for col in df.select_dtypes(include='object'):

    df[col] = le.fit_transform(df[col])


# Target column (adjust name if different)

target_col = "Credit_Score"

X = df.drop(target_col, axis=1)

y = df[target_col]


# -----

# FEATURE SCALING

# -----

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

```

```

# -----

# PCA

# -----

pca = PCA(n_components=0.92) # keep 92% variance
X_pca = pca.fit_transform(X_scaled)


# Plot explained variance

plt.figure(figsize=(8,5))

plt.plot(np.cumsum(pca.explained_variance_ratio_), marker='o')

plt.xlabel("Number of Components")

plt.ylabel("Cumulative Explained Variance")

plt.title("PCA Explained Variance")

plt.grid()

plt.show()


# -----

# TRAIN TEST SPLIT

# -----

X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.2, random_state=42)


# -----

# MODELS

# -----

models = {

    "Logistic Regression": LogisticRegression(max_iter=500),

```

```

"Decision Tree": DecisionTreeClassifier(),
"Random Forest": RandomForestClassifier(),
"KNN": KNeighborsClassifier(),
"Naive Bayes": GaussianNB()
}

results = {}

# -----
# TRAIN & EVALUATE
# -----

for name, model in models.items():

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    results[name] = [acc, prec, rec, f1]

    print(f"\n{name}")
    print("Accuracy:", acc)
    print("Precision:", prec)
    print("Recall:", rec)

```

```

print("F1 Score:", f1)

# -----
# MODEL COMPARISON
# -----

results_df = pd.DataFrame(results, index=["Accuracy", "Precision", "Recall", "F1 Score"]).T

results_df.plot(kind="bar", figsize=(10,6))

plt.title("Model Performance Comparison")

plt.xticks(rotation=45)

plt.ylabel("Score")

plt.show()

# -----
# CREDIT SCORE DISTRIBUTION
# -----

plt.figure(figsize=(6,4))

sns.countplot(x=y)

plt.title("Credit Score Distribution")

plt.show()

```

RESULTS

Credit score distribution visualized.

PCA explained variance plotted.

Model comparison performed.

CONCLUSION

PCA reduced dimensionality effectively. Random Forest performed best followed by Decision Tree and Logistic Regression.